



United International University (UIU)
Department of Computer Science and Engineering (CSE)
Course: VLSI Design (CSE 4327)

Experiment 5: Design of Sequential Logic Circuits using Verilog

Objectives

1. To understand the difference between combinational and sequential logic circuits, and the role of a clock signal and a reset in synchronous design.
2. To design and simulate an edge-triggered D flip-flop in Verilog.
3. To design and simulate a JK flip-flop in Verilog, and to verify its four operating modes: hold, reset, set, and toggle.
4. To design and simulate a 4-bit synchronous up-counter built from edge-triggered sequential logic.
5. To verify each design using the Incisive Enterprise Simulator (IES) / NCLaunch flow (compile, elaborate, simulate) and to analyze the results using SimVision waveforms.

Introduction

Unlike a combinational circuit, whose output depends only on the present inputs, a sequential logic circuit has memory: its output depends on both the present inputs and the circuit's present state, which is stored in flip-flops and updated on clock edges. The flip-flop is the fundamental storage element of sequential design. A D (data) flip-flop simply captures the value present at its D input on the active clock edge and holds it until the next edge. A JK flip-flop is a more versatile storage element with two control inputs, J and K, that together select one of four operating modes on each clock edge: hold the current state, reset to 0, set to 1, or toggle to the opposite state. Counters are a classic application of flip-flops, where a chain of flip-flops updates its combined state by one count on every clock edge, wrapping back to zero once it reaches its maximum value.

In this experiment, Verilog HDL is used to design and simulate a D flip-flop, a JK flip-flop, and a 4-bit synchronous up-counter, each with an active-low asynchronous reset ($\text{rst} = 0$ forces the state to zero regardless of the clock). As in the previous experiments, the Cadence Incisive Enterprise Simulator (IES), invoked through NCLaunch, is used for compilation, elaboration, and simulation.

Note: This experiment assumes your Cadence tools environment (cshrc/csh) has already been configured, as covered in the earlier labs. If you are starting a fresh session, remember to switch to csh before invoking any Cadence tool.

Part 1: Designing and Verifying the D Flip-Flop

1. Create a lab directory and a simulation sub-directory under your work area:

```
mkdir lab4
cd lab4
mkdir simulation
```

cd simulation

2. Write Verilog code for a D flip-flop with an active-low asynchronous reset. Save the file as d_ff.v:

vi d_ff.v

```
`timescale 1ns/1ps

module d_ff (output reg q, input d, clk, rst);
    always @(posedge clk or negedge rst) begin
        if (!rst)
            q <= 0;
        else
            q <= d;
    end
endmodule
```

Listing 1: d_ff.v - edge-triggered D flip-flop with active-low asynchronous reset.

3. Write Verilog test bench code for the above D flip-flop. Save the file as d_ff_tb.v:

vi d_ff_tb.v

```
`timescale 1ns/1ps

module d_ff_tb;
    reg d, clk, rst;
    wire q;

    d_ff uut (q, d, clk, rst);

    always #5 clk = ~clk;

    initial begin
        clk = 0; rst = 0; d = 0;
        #10 rst = 1;
        #10 d = 1;
        #10 d = 0;
        #10 d = 1;
        #10 $finish;
    end

    initial
        $monitor("Time=%0t D=%b Q=%b", $time, d, q);
endmodule
```

Listing 2: d_ff_tb.v - test bench that generates a clock, releases reset, and applies a sequence of D values.

4. Simulate the Verilog files and verify the results. Include d_ff_tb.v and d_ff.v in the simulation.

Note: The sensitivity list ‘posedge clk or negedge rst’ makes the reset asynchronous: whenever rst falls to 0, q is cleared immediately, independent of the clock edge. On every rising edge of clk while rst is high, q simply takes the value of d.

Part 2: Designing and Verifying the JK Flip-Flop

5. Write Verilog code for a JK flip-flop with an active-low asynchronous reset. Save the file as jk_ff.v:

vi jk_ff.v

```

`timescale 1ns/1ps

module jk_ff (output reg q, output qbar, input j, k, clk, rst);
    assign qbar = ~q;

    always @(posedge clk or negedge rst) begin
        if (!rst)
            q <= 0;
        else begin
            case ({j, k})
                2'b00: q <= q;    // Hold
                2'b01: q <= 0;    // Reset
                2'b10: q <= 1;    // Set
                2'b11: q <= ~q;   // Toggle
            endcase
        end
    end
endmodule

```

Listing 3: *jk_ff.v* - JK flip-flop with active-low asynchronous reset, implemented with a case statement on {j, k}.

6. Write Verilog test bench code for the above JK flip-flop. Save the file as jk_ff_tb.v:

vi jk_ff_tb.v

```

`timescale 1ns/1ps

module jk_ff_tb;
    reg j, k, clk, rst;
    wire q, qbar;

    jk_ff uut (q, qbar, j, k, clk, rst);

    always #5 clk = ~clk;

    initial begin
        clk = 0; rst = 0; j = 0; k = 0;
        #10 rst = 1;
        #10 j = 1; k = 0;    // Set
        #10 j = 0; k = 0;    // Hold
        #10 j = 0; k = 1;    // Reset
        #10 j = 1; k = 1;    // Toggle
        #10 j = 1; k = 1;    // Toggle again
        #10 $finish;
    end

    initial
        $monitor("Time=%0t J=%b K=%b Q=%b Qbar=%b",
            $time, j, k, q, qbar);
endmodule

```

Listing 4: *jk_ff_tb.v* - test bench that exercises the set, hold, reset, and toggle modes of jk_ff in sequence.

7. Simulate the Verilog files and verify the results. Include jk_ff_tb.v and jk_ff.v in the simulation.

The truth table below summarizes the four operating modes selected by the J and K inputs on each active clock edge:

J	K	Q (next)	Operation
0	0	Q	Hold

J	K	Q (next)	Operation
0	1	0	Reset
1	0	1	Set
1	1	$\sim Q$	Toggle

Part 3: Designing and Verifying the 4-bit Counter

8. Write Verilog code for a 4-bit synchronous up-counter with an active-low asynchronous reset. The counter should count from 0000 to 1111, then wrap back to 0000. Save the file as counter_4bit.v:

vi counter_4bit.v

```
`timescale 1ns/1ps

module counter_4bit (output reg [3:0] count, input clk, rst);
    always @(posedge clk or negedge rst) begin
        if (!rst)
            count <= 4'b0000;
        else
            count <= count + 1'b1;
        end
    endmodule
```

Listing 5: counter_4bit.v - 4-bit synchronous up-counter with active-low asynchronous reset.

9. Write Verilog test bench code for the above counter. Save the file as counter_4bit_tb.v:

vi counter_4bit_tb.v

```
`timescale 1ns/1ps

module counter_4bit_tb;
    reg clk, rst;
    wire [3:0] count;

    counter_4bit uut (count, clk, rst);

    always #5 clk = ~clk;

    initial begin
        clk = 0;
        rst = 0;

        #10 rst = 1;    // Release reset
        #170 $finish;   // Run long enough to see the counter wrap around
    end

    initial
        $monitor("Time=%0t rst=%b count=%b (%0d)",
            $time, rst, count, count);
endmodule
```

Listing 6: counter_4bit_tb.v - test bench that releases reset and runs long enough (170 ns) to observe the counter wrap around from 1111 back to 0000.

10. Simulate the Verilog files and verify the results. Include counter_4bit_tb.v and counter_4bit.v in the simulation.

Note: With a 10 ns clock period, the counter advances once every 10 ns. Sixteen counts therefore take 160 ns to complete a full cycle; the test bench runs for 170 ns after reset is released so that the wrap-around from 1111 back to 0000 is clearly visible on the waveform.

Part 4: Launching NCLaunch and Creating the Work Library

The following steps walk through compiling, elaborating, and simulating a design using NCLaunch, using the JK flip-flop as the running example. Repeat the same procedure for the D flip-flop and the 4-bit counter designs at each ‘simulate and verify’ step above.

Note: Figures 1 to 8 below are reference screenshots carried over from Experiment 3 (Simulation of Multiplexers), since the NCLaunch/SimVision workflow is identical. Your own windows will show jk_ff.v / jk_ff_tb.v (and later d_ff.v / d_ff_tb.v, counter_4bit.v / counter_4bit_tb.v) in place of the mux21.v / mux21_test.v shown in these screenshots.

11. We will use the Incisive Enterprise Simulator (IES) to verify that each design is working properly. Invoke the tool by typing ‘nclaunch -new’ in the terminal:

nclaunch -new

12. The NCLaunch window will appear. Click on ‘Multiple Step’.

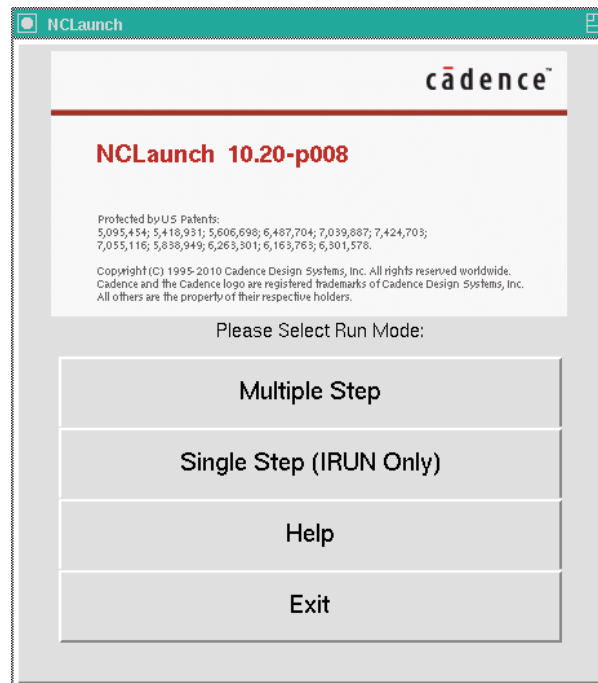


Figure 1: NCLaunch run-mode selection window.

13. The ‘nclaunch: Open Design Directory’ window will appear. Click on ‘Create cds.lib File...’.

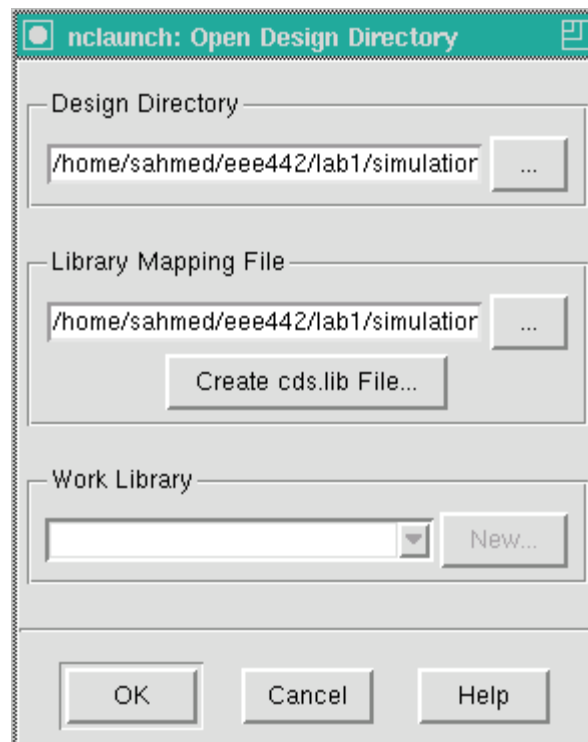


Figure 2: nclaunch Open Design Directory window, before the cds.lib file is created.

14. The 'Create a cds.lib file' window will appear. Click on 'Save'.

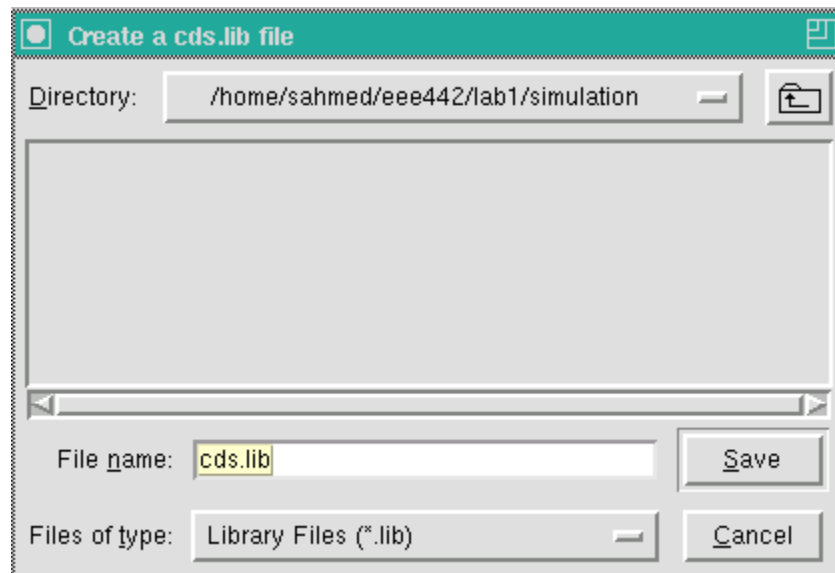


Figure 3: Create a cds.lib file window.

15. The 'New cds.lib File' window will appear. Select the third option, since we are using Verilog. Click on 'OK'.

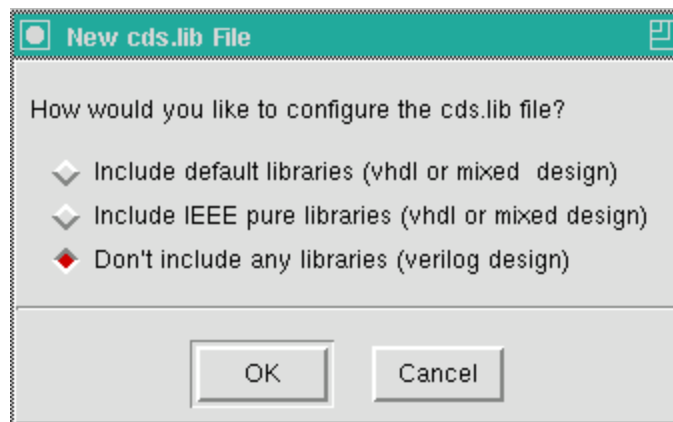


Figure 4: New cds.lib File window, with 'Don't include any libraries (Verilog design)' selected.

16. Click 'OK' in the 'nclaunch: Open Design Directory' window.

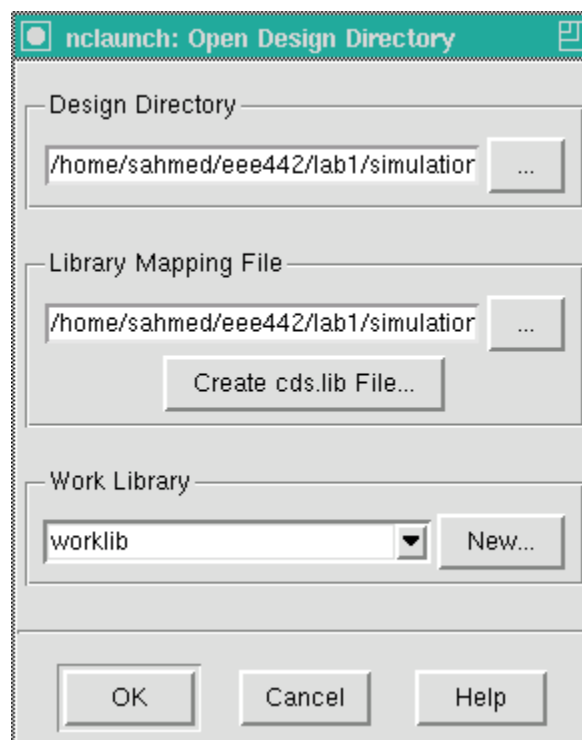


Figure 5: Open Design Directory window with the worklib now populated.

Part 5: Compiling the Design

17. In the NCLaunch window, we will see the design (jk_ff.v) and the test bench (jk_ff_tb.v) that we created in the simulation directory.

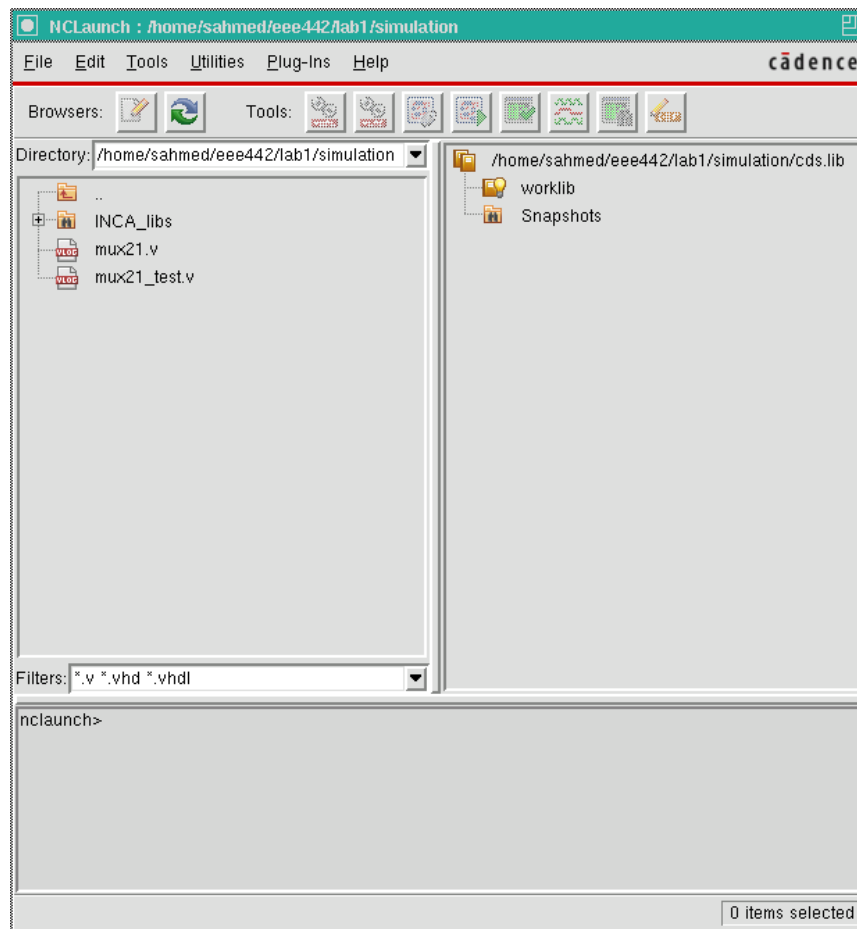


Figure 6: NCLaunch window showing the design and test bench files in the simulation directory (mux21.v / mux21_test.v shown here; you will see jk_ff.v / jk_ff_tb.v).

18. Now we will COMPILE the code to check its syntax and semantics. Select both the design and the test bench files, then click on ncvtlog for Verilog designs.

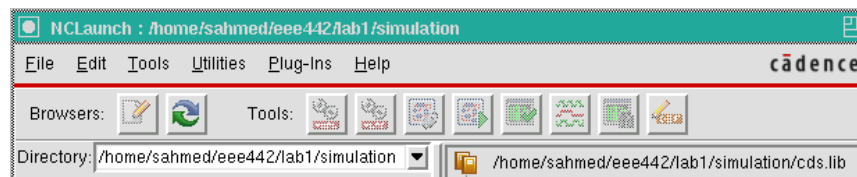


Figure 7: NCLaunch toolbar, with the ncvtlog (compile) icon highlighted.

19. Any error in the code will be reported in the 'Command Entry Window' at the bottom of NCLaunch. Clean all the errors in the design and/or the test bench before continuing.

Part 6: Elaboration - Building the Design Hierarchy

Elaboration constructs the design hierarchy and connects the signals between the design and the test bench.

20. Open the 'worklib' directory on the right side of the window and select the test bench, since it is the top module (it instantiates and contains the design module). Click on the 'launch elaborator' (ncelab).



Figure 8: NCLaunch toolbar, with the ncelab (elaborate) icon highlighted.

Part 7: Simulation - Running and Viewing the Waveforms

Simulation executes the elaborated design against the applied stimulus.

21. Open the 'Snapshots' directory on the right side of the window and select the worklib snapshot of the test bench. Click on 'launch simulator'.



Figure 9: NCLaunch toolbar, with the launch simulator icon highlighted.

22. A 'Console - SimVision' window will appear. This is used to perform simulation in command mode. Minimize this window, since we will be using GUI mode for simulation.
23. A 'Design Browser - SimVision' window also appears. There are many options in the Design Browser to analyze the design and for debugging. Select the test bench module on the left side of the window, then click on 'waveform'.

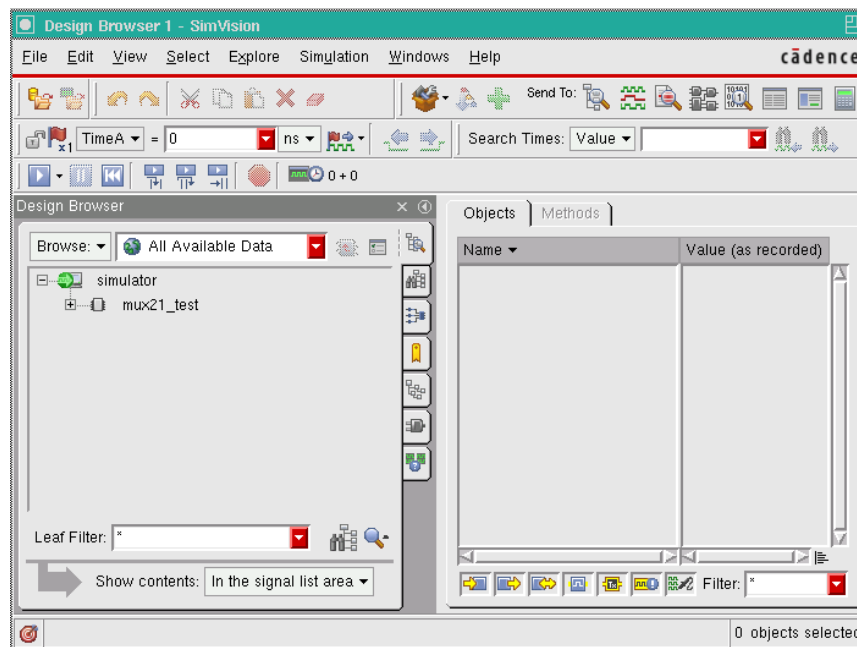


Figure 10: Design Browser - SimVision window, with the test bench module selected.

24. A 'Waveform - SimVision' window will appear. In the waveform window we will see the ports of the design (j, k, clk, rst, q, qbar for the JK flip-flop). Click on 'run simulation' to start the simulation. Use the 'pause' button to interrupt or stop the simulation. Use options such as zoom in and zoom out to analyze the plot; the Waveform window offers many further options for analyzing the design and debugging.

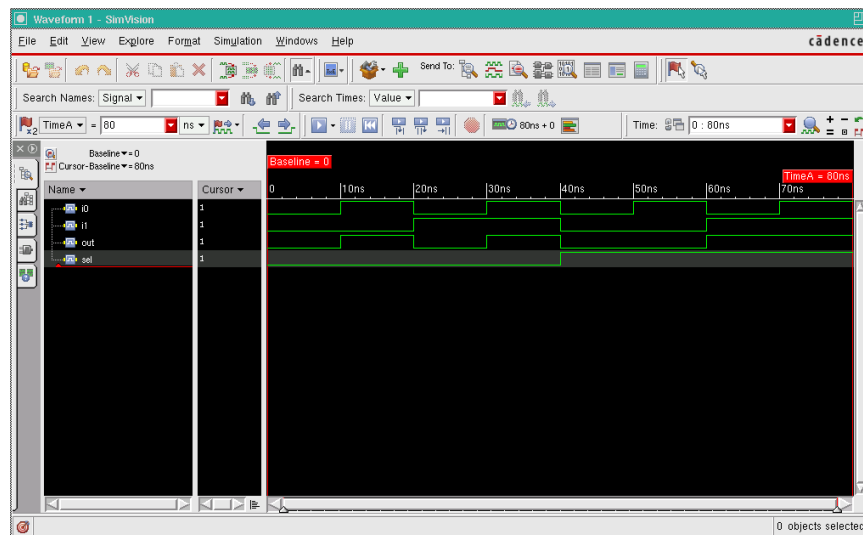


Figure 11: Waveform - SimVision window, showing simulated signals for the design under test.

25. After verifying the design, close the tools, and repeat Parts 4 to 7 for the remaining designs (D flip-flop and 4-bit counter).

Appendix: Complete Verilog Source Listings

For convenience, the complete source code for all three designs and their test benches is collected below.

1. D Flip-Flop

```
`timescale 1ns/1ps

module d_ff (output reg q, input d, clk, rst);
    always @(posedge clk or negedge rst) begin
        if (!rst)
            q <= 0;
        else
            q <= d;
    end
endmodule
```

d_ff.v

2. D Flip-Flop Test Bench

```
`timescale 1ns/1ps

module d_ff_tb;
    reg d, clk, rst;
    wire q;

    d_ff uut (q, d, clk, rst);

    always #5 clk = ~clk;

    initial begin
        clk = 0; rst = 0; d = 0;
        #10 rst = 1;
        #10 d = 1;
        #10 d = 0;
        #10 d = 1;
        #10 $finish;
    end

    initial
        $monitor("Time=%0t D=%b Q=%b", $time, d, q);
endmodule
```

d_ff_tb.v

3. JK Flip-Flop

```
`timescale 1ns/1ps

module jk_ff (output reg q, output qbar, input j, k, clk, rst);
    assign qbar = ~q;

    always @(posedge clk or negedge rst) begin
        if (!rst)
            q <= 0;
        else begin
            case ({j, k})
```

```

                2'b00: q <= q;    // Hold
                2'b01: q <= 0;    // Reset
                2'b10: q <= 1;    // Set
                2'b11: q <= ~q;   // Toggle
            endcase
        end
    end
endmodule

```

jk_ff.v

4. JK Flip-Flop Test Bench

```

`timescale 1ns/1ps

module jk_ff_tb;
    reg j, k, clk, rst;
    wire q, qbar;

    jk_ff uut (q, qbar, j, k, clk, rst);

    always #5 clk = ~clk;

    initial begin
        clk = 0; rst = 0; j = 0; k = 0;
        #10 rst = 1;
        #10 j = 1; k = 0; // Set
        #10 j = 0; k = 0; // Hold
        #10 j = 0; k = 1; // Reset
        #10 j = 1; k = 1; // Toggle
        #10 j = 1; k = 1; // Toggle again
        #10 $finish;
    end

    initial
        $monitor("Time=%0t J=%b K=%b Q=%b Qbar=%b",
            $time, j, k, q, qbar);
endmodule

```

jk_ff_tb.v

5. 4-bit Counter

```

`timescale 1ns/1ps

module counter_4bit (output reg [3:0] count, input clk, rst);
    always @(posedge clk or negedge rst) begin
        if (!rst)
            count <= 4'b0000;
        else
            count <= count + 1'b1;
    end
endmodule

```

counter_4bit.v

6. 4-bit Counter Test Bench

```
`timescale 1ns/1ps

module counter_4bit_tb;
    reg clk, rst;
    wire [3:0] count;

    counter_4bit uut (count, clk, rst);

    always #5 clk = ~clk;

    initial begin
        clk = 0;
        rst = 0;

        #10 rst = 1;    // Release reset
        #170 $finish;    // Run long enough to see the counter wrap around
    end

    initial
        $monitor("Time=%0t rst=%b count=%b (%0d)",
                 $time, rst, count, count);
endmodule
```

counter_4bit_tb.v

Lab Report / Submission Checklist

1. Verilog source code and test bench listings for the D flip-flop, JK flip-flop, and 4-bit counter.
2. A screenshot of the NCLaunch window showing successful compilation (no errors in the Command Entry Window) for each design.
3. Waveform screenshots from SimVision for each design, clearly showing clk, rst, and the relevant inputs/outputs (d/q for the D flip-flop; j, k, q, qbar for the JK flip-flop; count for the counter).
4. A short discussion (150-200 words) explaining the four operating modes of the JK flip-flop observed in simulation, and how the 4-bit counter's wrap-around from 1111 to 0000 was verified on the waveform.

Credits

This lab manual was prepared and formatted by:

Originated: EEE, UIU

Sudipto Mondal

Lecturer, Department of Computer Science and Engineering (CSE)

United International University (UIU)

Email: sudiptomondal@cse.uiu.ac.bd